

Reto Técnico iOS – Swift + SwiftUI

Reto Técnico iOS – Swift + SwiftUI

Nivel: Mid → Senior

Objetivo

Desarrollar una aplicación móvil iOS utilizando **Swift** y **SwiftUI**, aplicando buenas prácticas modernas, arquitectura limpia, principios SOLID y Swift Concurrency.

El código deberá almacenarse en un **repositorio público de GitHub**.

Requerimientos Funcionales

1. Consumo de API REST

Consumir **una** de las siguientes APIs públicas:

- <https://disneyapi.dev/>
- <https://ygoprodeck.com/api-guide/>

Requisitos:

- Implementar capa de red con `URLSession` (sin librerías de terceros para networking)
 - Manejo de errores con tipos de error propios (enum que conforme a `Error`)
 - Llamadas asíncronas con **`async/await`**
-

2. Listado Principal (Home)

Al ingresar a la aplicación se deberá mostrar:

- Listado de elementos obtenidos desde la API
 - Diseño moderno usando **SwiftUI** (no UIKit, no Storyboards)
 - Imagen cargada de forma lazy con caché en memoria
-

3. Pantalla de Detalle

Al tocar un elemento del listado:

- Navegar a pantalla de detalle con información completa del elemento
 - Uso de **NavigationStack** con rutas type-safe (`NavigationPath` o enum de rutas)
-

4. Persistencia Offline (Modo Sin Conexión)

Los datos obtenidos del servicio deben:

- Persistirse localmente usando **SwiftData** (iOS 17+) o **Core Data**
 - Estar disponibles sin conexión a Internet
 - Implementar estrategia **offline-first**: mostrar datos locales mientras carga la red
-

5. Sistema de Favoritos

- El usuario podrá marcar/desmarcar elementos como favoritos
 - Los favoritos deberán persistirse localmente
 - Crear una pestaña o pantalla independiente para visualizar favoritos
 - La información debe sobrevivir al cierre de la app
-

Requerimientos Técnicos Obligatorios

Arquitectura

- **MVVM** estricto
- **Clean Architecture** con 3 capas:
 - Capa de presentación (View + ViewModel)
 - Capa de dominio (UseCases + Entities + interfaces/protocolos de repositorio)
 - Capa de datos (Repository impl + DataSource remota/local)

Manejo de Estado

- `@Observable` (iOS 17+) o `ObservableObject` + `@StateObject` / `@EnvironmentObject`
- **Combine** o **async/await** + **AsyncStream** para flujos reactivos

Concurrencia

- Swift Concurrency obligatorio: `async/await`, `Task`, `@MainActor`
- Manejo correcto de contextos de hilo (no bloquear main thread)
- Cancelación de tareas en `onDisappear`

Inyección de Dependencias

- Inyección por protocolo (sin frameworks externos obligatorio)
- DIContainer o factory propio (Swinject permitido como extra)

UI

- 100% **SwiftUI** (NO UIKit, NO Storyboards, NO XIBs)
- Uso de:
 - List / LazyVStack para el listado
 - NavigationStack para navegación
 - Componentes reutilizables (ViewModifier, ViewBuilder, vistas extraídas)
 - Estados de carga: loading / error / empty / success

Testing (*requerido para nivel Sr*)

- Mínimo **2 Unit Tests** sobre casos de uso o ViewModel
 - Uso de protocolos/mocks para aislar dependencias
 - XCTest framework
-

Criterios de Evaluación

Criterio	Peso
Arquitectura y separación de capas	Alto
Uso correcto de Swift Concurrency	Alto
Calidad y limpieza del código	Alto
Funcionamiento offline y favoritos	Medio
UI/UX y uso de SwiftUI	Medio
Tests unitarios	Medio (obligatorio Sr)
Naming, organización de archivos	Medio

Diferenciadores Senior

Se valorará positivamente:

- Uso de actor para acceso thread-safe a datos compartidos
 - Manejo de URLCache para caché HTTP
 - Paginación desde la API
 - Accesibilidad básica (.accessibilityLabel, .accessibilityHint)
 - CI básico (GitHub Actions que compile el proyecto)
-

Entregables

El repositorio debe contener:

Código Fuente

- Proyecto funcional compilable en Xcode 15+, iOS 16+ mínimo
- Buen naming y organización por features o capas

README.md Completo

Debe incluir:

- Descripción del proyecto
 - Arquitectura utilizada (diagrama o descripción de capas)
 - Librerías/frameworks utilizados
 - Requisitos mínimos (versión Xcode, iOS target)
 - Instrucciones para correr el proyecto
 - Decisiones técnicas relevantes (¿por qué SwiftData vs Core Data? ¿por qué URLSession sin Alamofire?)
-

Stack Técnico Esperado

Categoría	Tecnología
Lenguaje	Swift 5.9+
UI	SwiftUI
Networking	URLSession + async/await
Persistencia	SwiftData / Core Data
Arquitectura	MVVM + Clean Architecture
Concurrencia	Swift Concurrency
Testing	XCTest
Mínimo iOS	16.0
Xcode	15+